

Ascend C 编程模型与编程范式

目 录

1 引言：为什么需要理解编程模型	2
2 AI Core 硬件架构	2
2.1 计算单元与搬运单元	2
2.2 存储层次与数据通路	3
3 SPMD 编程模型与流水线并行	4
3.1 SPMD：单程序多数据	4
3.2 流水线任务：Stage 与 Progress	4
4 Ascend C 语法扩展	5
4.1 类库 API 总览	5
4.2 多级 API：基础与高阶	5
4.3 GlobalTensor：全局数据	6
4.4 LocalTensor：片上数据	6
4.5 TPipe：内存管理	6
4.6 TQue：通信与同步	7
4.7 四步操作：AllocTensor、EnQue、DeQue、FreeTensor	7
5 向量编程范式：实战 KernelAdd	8
5.1 算子分析与切分策略	8
5.2 核函数声明与封装	8
5.3 算子类实现	8
6 矩阵编程范式：实战 Matmul	10
6.1 矩阵乘法与数据切分	10
6.2 多核切分与核内切分	10
6.3 Matmul 高阶 API 与 kernel 侧开发	11
6.4 核函数声明与 host 侧 Tiling	12
7 混合编程范式：Matmul + LeakyReLU	13
7.1 融合算子的优势	13
7.2 Process 三阶段实现	13
8 更多 Ascend C 算子样例	15
9 本章你将学会	15
10 要点速查	15
11 小结	16

1 引言：为什么需要理解编程模型

当深度学习框架自带的算子无法满足需求时, 要么框架不包含某个算子, 要么已有算子性能不够理想, 开发者需要直接在硬件层面编写高效算子。**Ascend C** 提供了面向昇腾 AI Core 的编程模型和编程范式, 让开发者能够充分发挥硬件算力。理解编程模型, 是写出正确且高效算子的前提。

直觉 不妨把 AI Core 想象成一座工厂: 有负责调度指挥的“车间主任” (标量计算单元), 有专做重体力活的“壮汉” (矩阵计算单元), 有灵巧的“技术工人” (向量计算单元), 还有负责搬运原料和成品的“物流团队” (DMA 搬运单元)。Ascend C 的编程模型, 就是这座工厂的“操作规程”, 告诉每个角色什么时候做什么, 如何协调配合。



Figure 1: AI Core 工厂类比的视觉引入 (AI 生成装饰图, 不表达具体硬件结构)

本章我们将从 AI Core 硬件架构出发, 理解 **SPMD** (单程序多数据) 模型与流水线并行; 学习 Ascend C 的语法扩展, 包括 **Tensor** (张量) 存储抽象、**TPipe** (内存管理) 和 **TQue** (队列同步); 然后通过向量、矩阵、混合三种编程范式的实战, 掌握自定义算子开发的全流程。

2 AI Core 硬件架构

2.1 计算单元与搬运单元

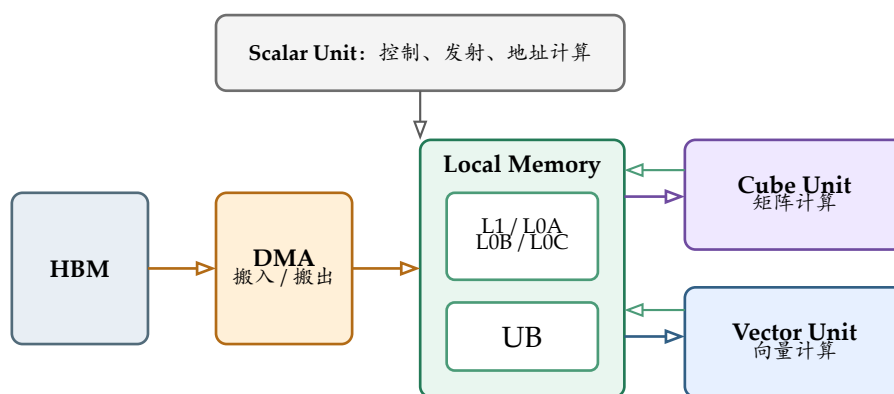
昇腾 AI Core 是昇腾处理器的核心计算单元, 其内部包含多种计算和搬运部件, 它们可以并行工作, 这是流水线编程范式的硬件基础。

1. **标量计算单元 (Scalar Unit):** 执行地址计算、循环控制、分支判断等标量逻辑, 类似 CPU 的标量核心, 负责计算流程控制、指令发射和地址计算。

2. **向量计算单元 (Vector Unit):** 负责张量数据的 SIMD 向量运算, 如逐元素加法、激活函数等。
3. **矩阵计算单元 (Cube Unit):** 执行矩阵乘法 (GEMM) 运算, 是 AI Core 算力的核心来源。
4. **DMA 搬运单元:** 负责数据在 Global Memory 和 Local Memory 之间的搬运, 内部包含搬入单元和搬出单元。

部件	功能	典型算子
Scalar Unit	标量逻辑、控制流、指令发射	循环计数、地址计算
Vector Unit	SIMD 向量运算	Add、Relu、Softmax
Cube Unit	矩阵乘法	Matmul、Conv2D
DMA Unit	数据搬运	DataCopy (搬入/搬出)

Figure 2 只保留本章后续编程会直接用到的宏观关系。它没有复刻原稿第 5 页的全部微架构部件, 因此不能当作 910B 的完整框图。



依据课程原稿第 5 页的 910B 架构抽象重绘: CeTZ 0.5.2

Figure 2: AI Core 的计算单元、搬运单元与数据通路

直觉 当 Cube 单元在计算矩阵乘法时, DMA 单元可以同时搬运下一批数据, Vector 单元可以做激活函数运算。这种“边搬边算”的并行能力, 正是流水线编程范式的硬件基础。

2.2 存储层次与数据通路

AI Core 内部有多级存储层次, 数据的处理遵循一条基本通路:

1. DMA 搬入单元把数据从 **Global Memory** (全局内存, 即 HBM) 搬运到 **Local Memory** (本地内存)。
2. 向量或矩阵计算单元从 Local Memory 读取数据, 完成计算后写回 Local Memory。
3. DMA 搬出单元把处理好的数据搬运回 Global Memory。

片上存储包含多个缓冲区: **L1 Buffer**、**L0A Buffer**、**L0B Buffer**、**L0C Buffer** 以及 **UB** (Unified Buffer, 统一缓冲区)。矩阵计算使用 L1/L0A/L0B/L0C 缓冲区, 向量计算使用 UB 缓冲区。

开发者无需直接操作物理存储地址。Ascend C 通过 *TPosition*（逻辑存储位置）抽象了存储级别, 用 *VECIN*、*VECOUT*、*VECCALC* 等逻辑位置代替物理存储概念, 屏蔽了底层硬件细节。

3 SPMD 编程模型与流水线并行

3.1 SPMD: 单程序多数据

SPMD (Single Program Multiple Data, 单程序多数据) 是 Ascend C 的核心执行模型。当算子程序被调用时, 运行时启动 N 个实例, 每个实例称为一个 **block**。所有 block 执行相同的代码, 拥有相同的参数, 它们之间唯一的区别是运行时实例 ID 不同, 即 `block_idx` 的值不同。

```
__aicore__ inline void Process() {  
    uint32_t lengthPerBlock = totalLength / GetBlockNum();  
    uint32_t offset = lengthPerBlock * GetBlockIdx();  
    // 每个 block 处理 [offset, offset + lengthPerBlock) 范围的数据  
}
```

`GetBlockIdx()` 返回当前 block 的编号, 类似 CUDA 中的 `threadIdx`。`GetBlockNum()` 返回 block 总数。通过合理的 Tiling 切分, 每个 block 独立处理互不重叠的数据分片, 实现数据并行。

直觉 想象一条流水线上有 N 个工人, 每个人拿着同一份操作手册 (同一份代码), 但各自负责不同的产品 (不同的数据分片)。工人的编号 (`block_idx`) 决定了他从哪个位置开始取料。

3.2 流水线任务: Stage 与 Progress

流水线任务 (Stage) 指的是单核处理程序中主程序调度的并行任务。在核函数内部, 可以通过流水线任务实现数据的并行处理来提升性能。

例如, 单核处理程序的功能可以被拆分成 3 个流水线任务: Stage1、Stage2、Stage3, 每个任务专注于完成单一功能。需要处理的数据被切分成 n 片, 用 Progress1 到 Progress n 表示, 每个任务需要依次完成 n 个数据切片的处理。

对于同一片数据, Stage1、Stage2、Stage3 之间存在依赖关系, 需要串行处理; 对于不同的数据切片, 同一时间点可以有多个 Stage 并行处理, 由此达到任务并行、提升性能的目的。

Figure 3 表达的是单个切片的数据依赖。连续输入多个切片后, 三个 Stage 才能在不同切片上同时工作。



依据课程原稿第 8 至 9 页重绘; Fletcher 0.5.8

Figure 3: 同一切片串行依赖、不同切片流水并行

例 假设有 3 个 Stage 和 3 个数据切片 (Progress 1, 2, 3), 每个 Stage 处理一片数据需要 1 个时间单位:

时间	Stage1 (CopyIn)	Stage2 (Compute)	Stage3 (CopyOut)
T1	Progress 1	-	-
T2	Progress 2	Progress 1	-
T3	Progress 3	Progress 2	Progress 1
T4	-	Progress 3	Progress 2
T5	-	-	Progress 3

不用流水线需要 $3 \times 3 = 9$ 个时间单位; 用流水线只需 5 个时间单位, 加速比 $\frac{9}{5} = 1.8$ 倍。这就是流水线并行的威力。

4 Ascend C 语法扩展

4.1 类库 API 总览

Ascend C 算子采用标准 C++ 语法和一组类库 API 进行编程。基本 API 分为三类:

1. **计算 API:** 向量计算 API 和矩阵计算 API, 如 Add、Mul、Matmul 等。
2. **搬运 API:** 数据搬运 API, 如 DataCopy, 负责 Global Memory 与 Local Memory 之间的数据传输。
3. **同步 API:** 队列操作 API 和内存管理 API, 如 EnQue、DeQue、AllocTensor、FreeTensor。

Ascend C 类库 API 采用类 SIMD 方式, 计算操作数都是 **Tensor** (张量) 类型, 包括 **GlobalTensor** 和 **LocalTensor**。

4.2 多级 API: 基础与高阶

为了降低开发者的使用门槛, Ascend C 将 API 分为 **基础 API** 和 **高阶 API** 两个层级。基础 API 对数据处理更偏向底层, 高阶 API 功能性更强、封装更好。多层级 API 封装的作用包括: 降低复杂指令的使用难度, 提供跨代兼容性保障, 同时保留最大灵活性。

指令类别	典型操作
单目指令	Exp、Ln、Abs、Sqrt、Relu、Sigmoid、Tanh
双目指令	Add、Sub、Mul、Div、Max、Min、And、Or
标量双目指令	Adds、Muls、Maxs、Mins、LeakyRelu
标量三目指令	Axpy
比较指令	Compare
选择指令	Select、ReduceV2
精度转换指令	Cast
规约指令	ReduceMax、ReduceMin、ReduceSum
特殊规约指令	WholeReduce、BlockReduce、PairReduce
数据转换操作	Transpose、TransDataTo5HD
数据填充操作	Duplicate、Brcb

指令类别	典型操作
数据搬移操作	DataCopy、Copy

4.3 GlobalTensor: 全局数据

GlobalTensor 用来存放 Global Memory（全局内存, 即 HBM）上的数据。核函数的输入输出数据通常存储在此。通过 `SetGlobalBuffer` 方法设置全局数据的指针和大小:

```
template <typename T> class GlobalTensor {
    void SetGlobalBuffer(__gm__ T* buffer, uint32_t bufferSize);
}
```

使用示例:

```
void Init(__gm__ uint8_t * __restrict__ src_gm,
          __gm__ uint8_t * __restrict__ dst_gm) {
    uint32_t dataSize = 256;
    GlobalTensor<int32_t> inputGlobal;
    // 设置源操作数在 Global Memory 上的起始地址为 src_gm
    // 大小为 256 个 int32_t
    inputGlobal.SetGlobalBuffer(
        reinterpret_cast<__gm__ int32_t *>(src_gm), dataSize);
    LocalTensor<int32_t> inputLocal = inQueueX.AllocTensor<int32_t>();
    // 将 Global Memory 上的 inputGlobal 拷贝到 Local Memory 的 inputLocal
    DataCopy(inputLocal, inputGlobal, dataSize);
}
```

4.4 LocalTensor: 片上数据

LocalTensor 用于存放 AI Core 中 Local Memory（本地内存）的数据。计算前需要将数据从 Global 搬运到 Local。LocalTensor 提供以下常用方法:

```
template <typename T> class LocalTensor {
    T GetValue(const uint32_t offset) const;
    template <typename T1> void SetValue(const uint32_t offset, const T1 value)
const;
    LocalTensor operator[](const uint32_t offset) const;
    uint32_t GetSize() const;
    void SetUserTag(const TTagType tag);
    TTagType GetUserTag() const;
};
```

`GetValue` 获取指定偏移处的值, `SetValue` 设置指定偏移处的值, `operator[]` 获取偏移后的新 LocalTensor, `GetSize` 返回当前 Tensor 的大小, `SetUserTag` 和 `GetUserTag` 用于自定义标签信息。

4.5 TPipe: 内存管理

任务间数据传递使用到的内存统一由内存管理模块 **Pipe** 进行管理。**TPipe** 作为片上内存管理者, 通过 `InitBuffer` 接口对外提供 Queue 内存初始化功能, 开发者可以通过该接口为指定的 Queue 分配内存。

```
private:
    TPipe pipe;
```



```
pipe.InitBuffer(que, num, len);
```

InitBuffer 接受三个参数: 队列对象 que, 内存块数 num, 每块大小 len (单位为 Bytes)。例如 pipe.InitBuffer(que, 4, 1024) 分配 4 块内存, 每块 1024 字节。

4.6 TQue: 通信与同步

TQue 完成任务之间的数据通信和同步。TQue 管理不同层级的物理内存时, 用一种抽象的逻辑位置 TPosition (也称 QuePosition) 来表达各级别存储, 代替了片上物理存储的概念, 开发者无须感知硬件架构。

1. 矢量 Queue: VECIN (输入队列)、VEECOUT (输出队列)、VECCALC (计算队列)。
2. 矩阵 Queue: A1、A2、B1、B2、CO1、CO2。

声明 Queue 的方式:

```
private:
    TQue<QuePosition::VECIN, BUFFER_NUM> inQueueX;
    TQue<QuePosition::VEECOUT, BUFFER_NUM> outQueueZ;
```

4.7 四步操作: AllocTensor、EnQue、DeQue、FreeTensor

TQue 队列的使用遵循固定的四步操作, 构成完整的 生产者与消费者 模型:

```
TPipe pipe;
TQue<TPosition::VEECOUT, 4> que;
pipe.InitBuffer(que, 4, 1024);

// 步骤 1: AllocTensor 从队列分配一个空闲 buffer
LocalTensor<half> tensor1 = que.AllocTensor<half>();

// 步骤 2: 填充数据后, EnQue 将 buffer 放入队列供下游消费
que.EnQue(tensor1);

// 步骤 3: DeQue 从队列取出一个已就绪的 buffer
LocalTensor<half> tensor1 = que.DeQue<half>();

// 步骤 4: 使用完毕后, FreeTensor 将 buffer 归还
que.FreeTensor(tensor1);
```

直觉 把 TQue 想象成一条传送带: AllocTensor 是工人取一个空箱子, EnQue 是把装好货物的箱子放上传送带, DeQue 是下游工人从传送带上取下箱子, FreeTensor 是用完后把空箱子还回去。这四步保证了流水线的正确同步。

当 BUFFER_NUM 设置为 2 时, 队列拥有两个 buffer, 天然支持 **Double Buffer** (双缓冲) 流水线: 当一个 buffer 正在被计算单元使用时, DMA 可以同时搬运另一个 buffer 的数据, 实现搬运与计算的重叠。

5 向量编程范式：实战 KernelAdd

5.1 算子分析与切分策略

开发自定义算子的流程分为三步：算子分析、核函数定义与封装、算子类实现。算子分析是前置任务，负责明确算子的输入输出、API 接口等需求；核函数定义和封装是编程的第一步；算子类实现是核心计算逻辑，分为内存初始化、数据搬入、计算逻辑、数据搬出四个部分。

以向量加法算子 `add_custom`（计算 $z = x + y$ ）为例，其数据切分策略如下：

1. 数据整体长度 `TOTAL_LENGTH` 为 $8 \times 2048 = 16384$ 。
 2. 平均分配到 8 个核上运行，单核处理数据大小 `BLOCK_LENGTH` 为 2048。
 3. 对于单核上的处理数据，切分成 8 块（不意味着 8 块就是性能最优）。
- 切分后的每个数据块再次切分成 2 块，即可开启 `double buffer`。此时每个数据块的长度 `TILE_LENGTH` 为 128 个数据。

例 以单核为例，`BLOCK_LENGTH = 2048`，`TILE_LENGTH = 128`，则 `tileNum = 2048 / 128 = 16`，即每个核需要循环处理 16 次。`BUFFER_NUM = 2` 表示开启 `double buffer`，队列中有 2 个缓冲区交替使用。总数据量 16384 个元素被 8 个核并行处理，每核 2048 个，每次搬运 128 个。

5.2 核函数声明与封装

核函数使用 `__global__` 函数类型限定符标识为核函数，可以被 `<<<...>>>` 调用；使用 `__aicore__` 标识该核函数在设备端 AI Core 上执行。入参统一使用 `GM_ADDR` 宏修饰：

```
extern "C" __global__ __aicore__ void add_custom(
    GM_ADDR x, GM_ADDR y, GM_ADDR z) {
    KernelAdd<half> op;
    op.Init(x, y, z);
    op.Process();
}
```

对核函数进行封装，得到 `add_custom_do` 函数，便于主程序调用：

```
void add_custom_do(uint32_t blockDim, void* l2ctrl, void* stream,
    uint8_t* x, uint8_t* y, uint8_t* z) {
    add_custom<<<blockDim, l2ctrl, stream>>>(x, y, z);
}
```

5.3 算子类实现

算子类 `KernelAdd` 包含三个流水线任务：`CopyIn`（搬入）、`Compute`（计算）、`CopyOut`（搬出）。类定义如下：

```
class KernelAdd {
public:
    __aicore__ inline KernelAdd() {}
    __aicore__ inline void Init(GM_ADDR x, GM_ADDR y, GM_ADDR z) {}
    __aicore__ inline void Process() {}
private:
    __aicore__ inline void CopyIn(int32_t progress) {}
    __aicore__ inline void Compute(int32_t progress) {}
}
```



```

__aicore__ inline void CopyOut(int32_t progress) {}
private:
    TPipe pipe;
    TQue<QuePosition::VECIN, BUFFER_NUM> inQueueX, inQueueY;
    TQue<QuePosition::VEOUT, BUFFER_NUM> outQueueZ;
    GlobalTensor<half> xGm, yGm, zGm;
};

```

Init 函数设置 GlobalTensor 的内存地址（注意按 block_idx 偏移以实现多核切分），并通过 pipe.InitBuffer 为各 Queue 分配内存：

```

__aicore__ inline void Init(GM_ADDR x, GM_ADDR y, GM_ADDR z) {
    // 按当前核编号计算偏移，实现多核并行
    xGm.SetGlobalBuffer(
        (__gm__ half*)x + BLOCK_LENGTH * GetBlockIdx(), BLOCK_LENGTH);
    yGm.SetGlobalBuffer(
        (__gm__ half*)y + BLOCK_LENGTH * GetBlockIdx(), BLOCK_LENGTH);
    zGm.SetGlobalBuffer(
        (__gm__ half*)z + BLOCK_LENGTH * GetBlockIdx(), BLOCK_LENGTH);
    // 为输入输出 Queue 分配内存，单位为 Bytes
    pipe.InitBuffer(inQueueX, BUFFER_NUM, TILE_LENGTH * sizeof(half));
    pipe.InitBuffer(inQueueY, BUFFER_NUM, TILE_LENGTH * sizeof(half));
    pipe.InitBuffer(outQueueZ, BUFFER_NUM, TILE_LENGTH * sizeof(half));
}

```

Process 函数循环执行 CopyIn, Compute, CopyOut 三阶段流水线：

```

__aicore__ inline void Process() {
    for (int32_t i = 0; i < tileNum; i++) {
        CopyIn(i);
        Compute(i);
        CopyOut(i);
    }
}

```

三个流水线任务分别完成搬入、计算、搬出：

```

__aicore__ inline void CopyIn(int32_t progress) {
    // 分配片上 x、y 数据空间
    LocalTensor<half> xLocal = inQueueX.AllocTensor<half>();
    LocalTensor<half> yLocal = inQueueY.AllocTensor<half>();
    // 从 Global Memory 拷贝到 Local Memory
    DataCopy(xLocal, xGm[progress * TILE_LENGTH], TILE_LENGTH);
    DataCopy(yLocal, yGm[progress * TILE_LENGTH], TILE_LENGTH);
    // 将填充好的数据放入输入队列
    inQueueX.Enqueue(xLocal);
    inQueueY.Enqueue(yLocal);
}

```

```

__aicore__ inline void Compute(int32_t progress) {
    // 从输入队列取出已就绪的数据
    LocalTensor<half> xLocal = inQueueX.DeQue<half>();
    LocalTensor<half> yLocal = inQueueY.DeQue<half>();
    // 分配输出数据空间

```

```

LocalTensor<half> zLocal = outQueueZ.AllocTensor<half>();
// 调用 Add API 执行矢量加法
Add(zLocal, xLocal, yLocal, TILE_LENGTH);
// 将结果放入输出队列
outQueueZ.Enqueue<half>(zLocal);
// 释放输入数据空间
inQueueX.FreeTensor(xLocal);
inQueueY.FreeTensor(yLocal);
}

__aicore__ inline void CopyOut(int32_t progress) {
    // 从输出队列取出结果
    LocalTensor<half> zLocal = outQueueZ.DeQueue<half>();
    // 从 Local Memory 拷贝回 Global Memory
    DataCopy(zGm[progress * TILE_LENGTH], zLocal, TILE_LENGTH);
    // 释放输出数据空间
    outQueueZ.FreeTensor(zLocal);
}

```

注意 *CopyIn* 中 *EnQueue* 在 *DataCopy* 之后, 确保数据填充完成才入队; *Compute* 中 *FreeTensor* 在 *EnQueue* 之后, 确保结果入队后才释放输入空间。这保证了流水线的正确同步。

6 矩阵编程范式：实战 Matmul

6.1 矩阵乘法与数据切分

Matmul (矩阵乘法) 的计算公式为 $C = A \times B + \text{Bias}$ 。其中 A 为左矩阵, 形状为 $[M, K]$; B 为右矩阵, 形状为 $[K, N]$; C 为结果矩阵, 形状为 $[M, N]$; Bias 为偏置, 形状为 $[1, N]$ 。

矩阵乘法的计算量远大于向量运算, 需要更精细的数据切分策略, 分为多核切分和核内切分两个层次。

6.2 多核切分与核内切分

多核切分: 将矩阵数据切分到不同核上并行处理。矩阵 A 沿 M 轴切分为多份 singleCoreM , 单核处理 $\text{singleCoreM} \times K$ 大小的数据; 矩阵 B 沿 N 轴切分为多份 singleCoreN , 单核处理 $K \times \text{singleCoreN}$ 大小的数据; 单核输出的矩阵 C 大小为 $\text{singleCoreM} \times \text{singleCoreN}$ 。

核内切分: 由于 Local Memory 通常无法完整容纳算子的输入与输出, 需要每次搬运一部分输入进行计算然后搬出, 再搬运下一部分。矩阵 A 沿 M 轴切分为 baseM , 沿 K 轴切分为 baseK ; 矩阵 B 沿 N 轴切分为 baseN , 沿 K 轴切分为 baseK 。每次计算 $\text{baseM} \times \text{baseN}$ 大小的 C 分块, 通过 K 轴累加完成。

例 令 $m = k = n = 32$, 输入数据类型为 half, 输出为 float32。矩阵 A 、 B 、 C 的形状均为 $[32, 32]$ 。假设使用单核 ($\text{singleCoreM} = \text{singleCoreN} = 32$), 核内切分设 $\text{baseM} = 16$, $\text{baseK} = 16$, $\text{baseN} = 16$, 则 M 方向切 2 块, N 方向切 2 块, K 方向切 2 块, 共需 $2 \times 2 \times 2 = 8$ 次分块计算。

6.3 Matmul 高阶 API 与 kernel 侧开发

Ascend C 提供一组 **Matmul 高阶 API**, 封装了常用的切分、数据搬运和计算逻辑, 方便用户快速实现矩阵乘法运算。算子类 MatmulKernel 的核心结构如下:

```
class MatmulKernel {
public:
    __aicore__ inline MatmulKernel() {};
    __aicore__ inline void Init(GM_ADDR a, GM_ADDR b, GM_ADDR bias,
                                GM_ADDR c, GM_ADDR workspace,
                                const TCubeTiling& tiling);
    __aicore__ inline void Process();
private:
    __aicore__ inline void CalcOffset(int32_t blockIdx,
                                       const TCubeTiling& tiling,
                                       int32_t& offsetA, int32_t& offsetB,
                                       int32_t& offsetC, int32_t& offsetBias);
    MatmulType<MatmulType<TPosition::GM, CubeFormat::ND, aType>,
               MatmulType<TPosition::GM, CubeFormat::ND, bType>,
               MatmulType<TPosition::LCM, CubeFormat::ND, cType>,
               MatmulType<TPosition::GM, CubeFormat::ND, biasType>> matmulObj;
    GlobalTensor<aType> aGlobal;
    GlobalTensor<bType> bGlobal;
    GlobalTensor<cType> cGlobal;
    GlobalTensor<biasType> biasGlobal;
    TPipe pipe;
    TCubeTiling tiling;
};
```

Init 函数设置各 GlobalTensor 的内存地址, 计算各核数据偏移, 并分配 workspace:

```
__aicore__ inline void Init(GM_ADDR a, GM_ADDR b, GM_ADDR bias,
                             GM_ADDR c, GM_ADDR workspace,
                             const TCubeTiling& tiling) {
    this->tiling = tiling;
    aGlobal.SetGlobalBuffer(
        reinterpret_cast<__gm__ aType*>(a), tiling.M * tiling.Ka);
    bGlobal.SetGlobalBuffer(
        reinterpret_cast<__gm__ bType*>(b), tiling.Kb * tiling.N);
    cGlobal.SetGlobalBuffer(
        reinterpret_cast<__gm__ cType*>(c), tiling.M * tiling.N);
    biasGlobal.SetGlobalBuffer(
        reinterpret_cast<__gm__ biasType*>(bias), tiling.N);

    int32_t offsetA = 0, offsetB = 0, offsetC = 0, offsetBias = 0;
    CalcOffset(GetBlockIdx(), tiling,
               offsetA, offsetB, offsetC, offsetBias);
    aGlobal = aGlobal[offsetA];
    bGlobal = bGlobal[offsetB];
    cGlobal = cGlobal[offsetC];
    biasGlobal = biasGlobal[offsetBias];

    SetSysWorkspace(workspace);
}
```

```

    if (GetSysWorkspacePtr() == nullptr) { return; }
}

```

Process 函数执行矩阵乘计算, 有两种计算方式: IterateAll 一次性计算全部, 或用 Iterate 循环逐个计算 (常用于还需要进一步处理数据的场景):

```

__aicore__ inline void Process() {
    REGIST_MATMUL_OBJ(&pipe, GetSysWorkspacePtr(), matmulObj);
    if (GetBlockIdx() >= 1) { return; }

    matmulObj.Init(&tiling);
    matmulObj.SetTensorA(aGlobal);
    matmulObj.SetTensorB(bGlobal);
    matmulObj.SetBias(biasGlobal);

    // 方式一: 一次性计算全部
    matmulObj.IterateAll(cGlobal);

    // 方式二: 循环逐个计算 (常用于还需后续处理)
    // while (matmulObj.Iterate()) {
    //     matmulObj.GetTensorC(cGlobal);
    // }

    matmulObj.End();
}

```

关键 API: REGIST_MATMUL_OBJ 注册 Matmul 对象, Init 初始化, SetTensorA / SetTensorB 设置左右矩阵, SetBias 设置偏置, IterateAll 执行完整计算, End 结束运算。

6.4 核函数声明与 host 侧 Tiling

核函数声明如下, 通过 GET_TILING_DATA 获取数据切分信息:

```

extern "C" __global__ __aicore__ void matmul_custom(
    GM_ADDR a, GM_ADDR b, GM_ADDR bias,
    GM_ADDR c, GM_ADDR workspace, GM_ADDR tiling) {
    GET_TILING_DATA(tilingData, tiling);
    MatmulKernel<half, half, float, float> matmulKernel;
    matmulKernel.Init(a, b, bias, c, workspace,
        tilingData.cubeTilingData);
    if (TILING_KEY_IS(1)) {
        matmulKernel.Process();
    }
}

```

host 侧需要创建 Tiling 结构体并实现 Tiling 函数。首先定义 Tiling 数据结构:

```

#include "register/tilingdata_base.h"
#include "tiling/tiling_api.h"
namespace optiling {
    BEGIN_TILING_DATA_DEF(MatmulCustomTilingData)
    TILING_DATA_FIELD_DEF_STRUCT(TCubeTiling, cubeTilingData);
    END_TILING_DATA_DEF;
    REGISTER_TILING_DATA_CLASS(MatmulCustom, MatmulCustomTilingData)
}

```

然后在 Tiling 函数中设置矩阵类型、形状和偏置, 调用 GetTiling 获取切分参数:

```
MatmulApiTiling cubeTiling(ascendcPlatform);
cubeTiling.SetAType(TPosition::GM, CubeFormat::ND,
                    matmul_tiling::DataType::DT_FLOAT16);
cubeTiling.SetBType(TPosition::GM, CubeFormat::ND,
                    matmul_tiling::DataType::DT_FLOAT16);
cubeTiling.SetCType(TPosition::LCM, CubeFormat::ND,
                    matmul_tiling::DataType::DT_FLOAT);
cubeTiling.SetBiasType(TPosition::GM, CubeFormat::ND,
                       matmul_tiling::DataType::DT_FLOAT);
cubeTiling.SetShape(M, N, K);
cubeTiling.SetOrgShape(M, N, K);
cubeTiling.SetBias(true);
cubeTiling.SetBufferSpace(-1, -1, -1);

MatmulCustomTilingData tiling;
if (cubeTiling.GetTiling(tiling.cubeTilingData) == -1) {
    return ge::GRAPH_FAILED;
}
```

7 混合编程范式: Matmul + LeakyReLU

7.1 融合算子的优势

昇腾 AI 处理器中, 矩阵计算单元 Cube 核与向量计算单元 Vector 核相互分离, 只通过 Global Memory 数据总线进行数据传递。在这种架构下, Cube 核与 Vector 核可以进行一定的并行计算。

融合算子 (Fused Operator) 即同时涉及矩阵计算和向量计算的算子, 正充分发挥了 Cube 核和 Vector 核分离的优势。例如 Matmul + LeakyReLU 融合算子, Cube 核负责矩阵乘法, Vector 核负责逐元素的非线性激活。

直觉 如果不做融合, Matmul 的结果需要先写回 Global Memory, 再由 Vector 核从 Global Memory 读取数据进行 LeakyReLU 计算, 这意味着两次额外的 HBM 读写。融合后, Matmul 的结果直接在片上内存中传递给 LeakyReLU, 省去了中间的 HBM 读写开销。这种 **Kernel Fusion** (核融合) 思想是后续大模型算子优化的核心策略。

7.2 Process 三阶段实现

matmul_leakyrelu 算子的 Process 部分分为三个阶段: MatmulCompute (矩阵乘)、LeakyReluCompute (激活)、CopyOut (搬出)。由于需要逐个处理数据, 使用 Iterate 循环而非 IterateAll:

```
__aicore__ inline void Process() {
    uint32_t computeRound = 0;
    REGIST_MATMUL_OBJ(&pipe, GetSysWorkspacePtr(), matmulObj);
    matmulObj.Init(&tiling);
    matmulObj.SetTensorA(aGlobal);
    matmulObj.SetTensorB(bGlobal);
    matmulObj.SetBias(biasGlobal);
```

```

// 逐轮计算：矩阵乘，激活，搬出
while (matmulObj.template Iterate<true>()) {
    MatmulCompute();
    LeakyReluCompute();
    CopyOut(computeRound);
    computeRound++;
}
matmulObj.End();
pipe_barrier(PIPE_ALL);
set_atomic_none();
}

三个阶段的实现:

__aicore__ inline void MatmulCompute() {
    // 分配输出空间，获取矩阵乘结果
    reluOutLocal = reluOutQueue_.AllocTensor<cType>();
    matmulObj.template GetTensorC<true>(reluOutLocal, false, true);
}

__aicore__ inline void LeakyReluCompute() {
    // 对矩阵乘结果执行 LeakyReLU 激活
    AscendC::LeakyRelu(reluOutLocal, reluOutLocal,
                       (cType)0.001, tiling.baseM * tiling.baseN);
    // 将结果放入输出队列
    reluOutQueue_.EnQue(reluOutLocal);
}

__aicore__ inline void CopyOut(uint32_t count) {
    // 从输出队列取出结果
    reluOutLocal = reluOutQueue_.DeQue<cType>();
    // 根据计算轮次确定输出偏移
    const uint32_t roundM = tiling.singleCoreM / tiling.baseM;
    const uint32_t roundN = tiling.singleCoreN / tiling.baseN;
    uint32_t startOffset = (count % roundM * tiling.baseM * tiling.N +
                           count / roundM * tiling.baseN);
    DataCopyParams copyParam = {
        (uint16_t)tiling.baseM,
        (uint16_t)(tiling.baseN * sizeof(cType) / DEFAULT_C0_SIZE),
        0,
        (uint16_t)((tiling.N - tiling.baseN) * sizeof(cType) /
                   DEFAULT_C0_SIZE)};
    AscendC::DataCopy(cGlobal[startOffset], reluOutLocal, copyParam);
    // 释放输出空间
    reluOutQueue_.FreeTensor(reluOutLocal);
}

```

LeakyRelu 的斜率参数设为 0.001。*CopyOut* 中根据 *computeRound* 计算输出偏移, 确保每轮结果写入 *Global Memory* 的正确位置。

8 更多 Ascend C 算子样例

除了上述三种编程范式, Ascend C 还支持更多复杂算子的实现:

1. **Sinh 算子**: 双曲正弦函数, 通过组合 Exp、Sub、Div 等基础向量 API 实现 $\sinh(x) = \frac{e^x - e^{-x}}{2}$ 。
2. **Strassen 算子**: 高性能矩阵乘法, 利用 Strassen 算法将矩阵乘的递归分治思想映射到 Ascend C 的多核与核内切分上, 减少乘法次数。
3. **LayerNorm 算子**: 层归一化, 涉及 ReduceMean、ReduceSum、Sqrt 等规约与向量 API 的组合, 是大模型中的常用算子。

这些样例展示了 Ascend C 在不同计算模式下的灵活性, 读者可以参考官方样例库深入学习。

9 本章你将学会

1. 描述 AI Core 的标量、向量、矩阵计算单元与 DMA 搬运单元的职责, 以及它们如何并行工作。
2. 解释 SPMD 模型中 block_idx 的作用, 以及流水线任务 Stage 与 Progress 的依赖与并行关系。
3. 使用 GlobalTensor、LocalTensor、TPipe、TQue 四个核心数据结构, 通过 AllocTensor、EnQue、DeQue、FreeTensor 四步操作管理数据流。
4. 按照向量编程范式 (Init, CopyIn, Compute, CopyOut) 实现一个完整的向量加法算子。
5. 使用 Matmul 高阶 API 实现矩阵乘法算子, 并理解多核切分与核内切分的策略。
6. 理解混合编程范式中 Kernel Fusion 的优势, 以及 Matmul + LeakyReLU 融合算子的实现方式。

10 要点速查

概念	要点
AI Core 部件	Scalar (控制)、Vector (向量)、Cube (矩阵)、DMA (搬运)
数据通路	Global Memory → DMA 搬入 → Local Memory → 计算 → DMA 搬出 → Global Memory
SPMD	多 block 执行同一份代码, block_idx 区分数据分片
GlobalTensor	全局内存数据, SetGlobalBuffer 设置地址
LocalTensor	片上内存数据, GetValue / SetValue / operator[]
TPipe	内存管理, InitBuffer(que, num, len)
TQue	队列同步, TPosition 抽象存储级别
四步操作	AllocTensor → EnQue → DeQue → FreeTensor
向量范式	Init → CopyIn → Compute → CopyOut
矩阵切分	多核: singleCoreM, singleCoreN; 核内: baseM, baseK, baseN
Matmul API	SetTensorA/B, SetBias, IterateAll, Iterate, End
混合范式	Cube + Vector 融合, Kernel Fusion 减少 HBM 读写

11 小结

本章深入介绍了 Ascend C 的编程模型与编程范式。我们从 AI Core 硬件架构出发, 理解了标量、向量、矩阵计算单元与 DMA 搬运单元如何并行工作, 以及从 Global Memory 到 Local Memory 再回到 Global Memory 的数据通路。SPMD 模型让同一份核函数在多个 block 上并行执行, 流水线任务通过 Stage 与 Progress 的重叠实现任务级并行。

在语法层面, Ascend C 通过 GlobalTensor 和 LocalTensor 统一管理数据, 通过 TPipe 管理片上内存, 通过 TQue 队列实现任务间通信与同步。AllocTensor、EnQue、DeQue、FreeTensor 四步操作构成了完整的生产者与消费者模型, 天然支持 Double Buffer 流水线。

在编程范式层面, 向量编程范式遵循 Init, CopyIn, Compute, CopyOut 四阶段流水线; 矩阵编程范式在多核切分和核内切分的基础上使用 Matmul 高阶 API; 混合编程范式将 Cube 和 Vector 融合在单个 Kernel 中, 通过 Kernel Fusion 减少 HBM 读写开销。这些编程范式是后续章节中算子开发和性能优化的基石。

讲义基于 Ascend C 编程模型与编程范式课程内容编写